

MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY

Permanently Affliated to JNTUH and Approved by AICTE, New Delhi

LECTURE NOTES

ON

EMBEDDED SYSTEMS DESIGN

IV B. Tech I semester (R22)
(2025-26)

Faculty Member

Mr. M. Ramanjaneyulu

Associate Professor ECE

MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY

IV Year B.Tech. ECE-II Sem

EMBEDDED SYSTEMS DESIGN (R22A0426)

COURSE OBJECTIVES:

For embedded systems, the course will enable the students to:

1. Understand the basics of an embedded system.
2. Program an embedded system.
3. To learn the design process of embedded system applications.
4. To understands the RTOS and inter-process communication.
5. To understand different communication interfaces.

UNIT-I INTRODUCTION TO EMBEDDED SYSTEMS

Definition of Embedded System, Embedded Systems Vs General Computing Systems, Classification, Major Application Areas, Purpose of Embedded Systems. Characteristics of embedded computing applications, challenges in embedded computing system design. The embedded system design process-requirements, specification, architecture design, designing hardware and software, components, system integration.

UNIT-II TYPICAL EMBEDDED SYSTEM

Core of the embedded system-general purpose and domain specific processors, ASICs, PLDs, COTs; Memory-ROM, RAM, memory according to the type of interface, memory selection for embedded systems; Sensors, actuators and other components, seven segment LED, relay, piezo buzzer, push button switch, reset circuit, brownout protection circuit, oscillator circuit real time clock, watch dog timer.

UNIT-III EMBEDDED FIRMWARE DESIGN AND DEVELOPMENT

Embedded firmware design approaches-super loop based approach, operating system based approach; Embedded firmware development languages-assembly language based development, high level language based development; Programming in embedded c.

UNIT-IV RTOS BASED EMBEDDED SYSTEM DESIGN

Operating system basics, types of operating systems, tasks, process and threads, multiprocessing and multitasking, task scheduling: non-preemptive and pre-emptive scheduling; task communication-shared memory, message passing.

UNIT-V COMMUNICATION INTERFACE

On board communication interfaces -I2C, SPI, UART,1-wire interface, parallel interface; External communication interfaces-RS232 and RS485, USB, infrared, Bluetooth, Wi-Fi, Zigbee, GPRS;

TEXT BOOKS:

1. Computers as Components –Wayne Wolf, Morgan Kaufmann (second edition).
2. Introduction to Embedded Systems - shibu k v, Mc Graw Hill Education.

REFERENCE BOOKS:

1. Embedded System Design -frank vahid, tony grivargis, john Wiley.
2. Embedded Systems- An integrated approach - Lyla b das, Pearson education 2012.
3. Embedded Systems – Raj Kamal, TMH
4. An embedded Software Primer, David e Simon, Pearson education

COURSE OUTCOMES:

Upon completion of this course, the students will be able to:

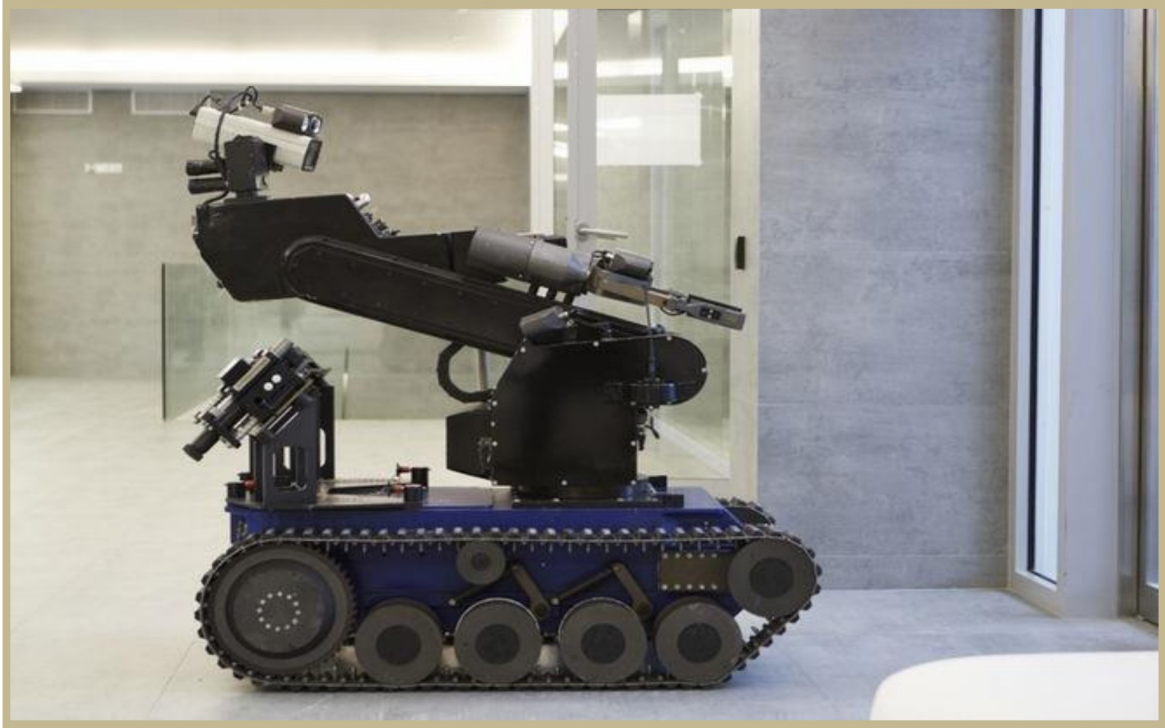
1. Understand and design the embedded systems
2. Learn the basics of OS and RTOS
3. Understand types of memory and interfacing to external world
4. Understand embedded firmware design approaches
5. Understand the various types of communication interfaces

EMBEDDED SYSTEM DESIGN

IV YEAR ECE

UNIT-I

INTRODUCTION TO EMBEDDED SYSTEMS



MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY
Permanently Affliated to JNTUH and Approved by AICTE, New Delhi

1. Introduction to Embedded Systems

What is Embedded System?

An Electronic/Electro mechanical system which is designed to perform a specific function and is a combination of both hardware and firmware (Software)

E.g. Electronic Toys, Mobile Handsets, Washing Machines, Air Conditioners, Automotive Control Units, Set Top Box, DVD Player etc…

Embedded Systems are:

- ❑
- ❑ Unique in character and behavior
- ❑ With specialized hardware and software

Embedded Systems Vs General Computing Systems:

General Purpose Computing System	Embedded System
A system which is a combination of generic hardware and General Purpose Operating System for executing a variety of applications	A system which is a combination of special purpose hardware and embedded OS for executing a specific set of applications
Contain a General Purpose Operating System (GPOS)	May or may not contain an operating system for functioning
Applications are alterable (programmable) by user (It is possible for the end user to re-install the Operating System, and add or remove user applications)	The firmware of the embedded system is pre-programmed and it is non-alterable by end-user
Performance is the key deciding factor on the selection of the system. Always „Faster is Better"	Application specific requirements (like performance, power requirements, memory usage etc) are the key deciding factors
Less/not at all tailored towards reduced operating power requirements, options for different levels of power management.	Highly tailored to take advantage of the power saving modes supported by hardware and Operating System
Response requirements are not time critical	For certain category of embedded systems like mission critical systems, the response time requirement is highly critical
Need not be deterministic in execution behavior	Execution behavior is deterministic for certain type of embedded systems like „Hard Real Time" systems

History of Embedded Systems:

- First Recognized Modern Embedded System: Apollo Guidance Computer (AGC) developed by Charles Stark Draper at the MIT Instrumentation Laboratory.
  - It has two modules
  - 1.Command module(CM) 2.Lunar Excursion module(LEM)
  - RAM size 256 , 1K ,2K words
  - ROM size 4K,10K,36K words
  - Clock frequency is 1.024MHz
  - 5000 ,3-input RTL NOR gates are used
  - User interface is DSKY(display/Keyboard)



- First Mass Produced Embedded System: *Autonetics **D-17** Guidance computer for Minuteman-I missile*

Classification of Embedded Systems:

- **Based on Generation** **(March-2017)**
- **Based on Complexity & Performance Requirements**
- **Based on deterministic behavior**
- **Based on Triggering**

1. Embedded Systems - Classification based on Generation

- **First Generation:** The early embedded systems built around 8-bit microprocessors like 8085 and Z80 and 4-bit microcontrollers

  EX. stepper motor control units, Digital Telephone Keypads etc.

  Second Generation: Embedded Systems built around 16-bit microprocessors and 8 or 16-bit microcontrollers, following the first generation embedded systems

  EX.SCADA, Data Acquisition Systems etc.

  Third Generation: Embedded Systems built around high performance 16/32 bit Microprocessors/controllers, Application Specific Instruction set processors like Digital Signal Processors (DSPs), and Application Specific Integrated Circuits (ASICs).The instruction set is complex and powerful.

  EX. Robotics, industrial process control, networking etc.

Fourth Generation: Embedded Systems built around System on Chips (SoC's), Re-configurable processors and multicore processors. It brings high performance, tight integration and miniaturization into the embedded device market

EX Smart phone devices, MIDs etc.

2. **Embedded Systems - Classification based on Complexity & Performance**

- **Small Scale:** The embedded systems built around low performance and low cost 8 or 16 bit microprocessors/ microcontrollers. It is suitable for simple applications and where performance is not time critical. It may or may not contain OS.

- **Medium Scale**: Embedded Systems built around medium performance, low cost 16 or 32 bit microprocessors / microcontrollers or DSPs. These are slightly complex in hardware and firmware. It may contain GPOS/RTOS.

- **Large Scale/Complex:** Embedded Systems built around high performance 32 or 64 bit RISC processors/controllers, RSoC or multi-core processors and PLD. It requires complex hardware and software. These system may contain multiple processors/controllers and co-units/hardware accelerators for offloading the processing requirements from the main processor. It contains RTOS for scheduling, prioritization and management.

3. **Embedded Systems - Classification Based on deterministic behavior:** It is applicable for Real Time systems. The application/task execution behavior for an embedded system can be either deterministic or non-deterministic

These are classified in to two types

1. **Soft Real time Systems:** Missing a deadline may not be critical and can be tolerated to a certain degree

2. **Hard Real time systems:** Missing a program/task execution time deadline can have catastrophic consequences (financial, human loss of life, etc.)

4. **Embedded Systems - Classification Based on Triggering:**

These are classified into two types

1. **Event Triggered :** Activities within the system (e.g., task run-times) are dynamic and depend upon occurrence of different events **.**

2. **Time triggered:** Activities within the system follow a statically computed schedule (i.e., they are allocated time slots during which they can take place) and thus by nature are predictable.

Major Application Areas of Embedded Systems:

- **Consumer Electronics:** Camcorders, Cameras etc.
- **Household Appliances:** Television, DVD players, washing machine, Fridge, Microwave Oven etc.
- **Home Automation and Security Systems:** Air conditioners, sprinklers, Intruder detection alarms, Closed Circuit Television Cameras, Fire alarms etc.
- **Automotive Industry:** Anti-lock breaking systems (ABS), Engine Control, Ignition Systems, Automatic Navigation Systems etc.
- **Telecom:** Cellular Telephones, Telephone switches, Handset Multimedia Applications etc.
- **Computer Peripherals:** Printers, Scanners, Fax machines etc.
- **Computer Networking Systems:** Network Routers, Switches, Hubs, Firewalls etc.
- **Health Care:** Different Kinds of Scanners, EEG, ECG Machines etc.
- **Measurement & Instrumentation:** Digital multi meters, Digital CROs, Logic Analyzers PLC systems etc.
- **Banking & Retail:** Automatic Teller Machines (ATM) and Currency counters, Point of Sales (POS)
- **Card Readers:** Barcode, Smart Card Readers, Hand held Devices etc.

Purpose of Embedded Systems: (DEC2016)

Each Embedded Systems is designed to serve the purpose of any one or a combination of the following tasks.

- Data Collection/Storage/Representation
- Data Communication
- Data (Signal) Processing
- Monitoring
- Control
- Application Specific User Interface

1. Data Collection/Storage/Representation:-

- Performs acquisition of data from the externalworld.
- The collected data can be either analog ordigital
- Data collection is usually done for storage, nalysis, manipulation and transmission
- The collected data may be stored directly in the system or may be transmitted to some other systems or it may be processed by the system or it may be deleted instantly after giving a meaningful representation

2. Data Communication:-

Embedded Data communication systems are deployed in applications ranging from complex satellite communicationsystems to simple home networking systems

Embedded Data communication systems are dedicated for datacommunication

The data communication can happen through a wired interface (like Ethernet, RS-232C/USB/IEEE1394 etc) or wireless interface (like Wi-Fi, GSM,/GPRS, Bluetooth, ZigBee etc)

Network hubs, Routers, switches, Modems etc are typical examples for dedicated data transmission embedded systems

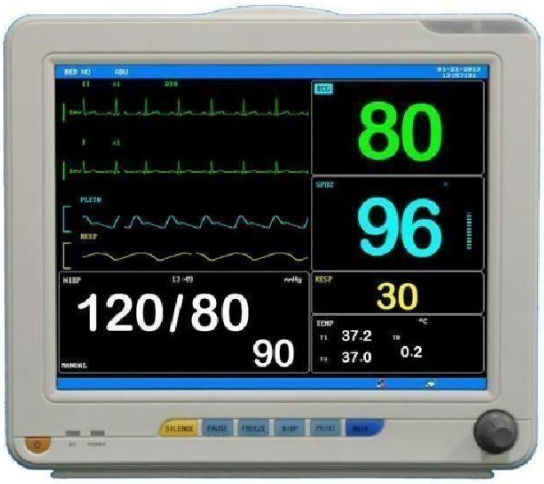
3. Data (Signal) Processing:-

Embedded systems with Signal processing functionalities are employed in applications demanding signal processing like Speech coding, synthesis, audio video codec, transmission applications etc

Computational intensive systems

Employs Digital Signal Processors (DSPs)

4. Monitoring:-



- Embedded systems coming under this category are specifically designed for monitoring purpose
- They are used for determining the state of some variables using input sensors
- They cannot impose control over variables.
- Electro Cardiogram (ECG) machine for monitoring the heart beat of a patient is a typical example for this
- The sensors used in ECG are the different Electrodes connected to the patient‟s body
- Measuring instruments like Digital CRO, Digital Multi meter, Logic Analyzer etc used in Control & Instrumentation applications are also examples of embedded systems for monitoring purpose

5. Control:-



- Embedded systems with control functionalities are used for imposing control over some variables according to the changes in input variables
- Embedded system with control functionality contains both sensors and actuators
- Sensors are connected to the input port for capturing the changes in environmental variable or measuring variable
- The actuators connected to the output port are controlled according to the changes in input variable to put an impact on the controlling variable to bring the controlled variable to the specified range

Air conditioner for controlling room temperature is a typical example for embedded system with „Control‟ functionality

Air conditioner contains a room temperature sensing element (sensor) which may be a thermistor and a handheld unit for setting up (feeding) the desired temperature

The air compressor unit acts as the actuator. The compressor is controlled according to the current room temperature and the desired temperature set by the end user.

6. Application Specific User Interface:-



Embedded systems which are designed for a specific application

Contains Application Specific User interface (rather than general standard UI) like key board, Display units etc

COMPLEX SYSTEM AND MICROPROCESSORS:

- **Three main tasks or components in embedded system design:**
  - Selecting and integrating hardware to give computer like functionalities
  - Dumping main application software generally into flash or ROM and the application software performs concurrently the number of tasks.
  - Integrating with a real time operating system (RTOS), this supervises the application software tasks running on the hardware and organizes the accesses to system resources according to priorities and timing constraints of tasks in the system.

Embedding Computers:

- **Whirlwind**, a computer designed at MIT in the late 1940s and early 1950s. Whirlwind was also the first computer designed to support ***real-time*** operation and was originally conceived as a mechanism for controlling an **aircraft simulator**. It was extremely large physically compared to today‘s computers(e.g., it contained over 4,000 vacuum tubes).
- Very-large-scale integration (VLSI) is the process of creating an integrated circuit (IC) by combining thousands of transistors into a single chip. VLSI began in the 1970s. A microprocessor is a single-chip CPU. Very large scale integration (VLSI) technology allowed us to put a complete CPU on a single chipsince 1970s, but those CPUs were very simple.

  In 1971 the **first microprocessor the Intel 4004 invented by Ted Hoff**, was designed for an embeddedapplication, namely, a calculator. The calculator was not a general-purpose computer—it merely provided basic arithmetic functions. The **HP-35 was the first handheld calculator** to perform transcendental functions. It was introduced in 1972, so it used several chips to implement the CPU,rather than a single-chip microprocessor.

- **Automobile designers** started making use of the microprocessor soon after single-chip CPUs became available. The most important and sophisticated use of microprocessors in automobiles was to**control the engine**: determining when spark plugs fire, controlling the fuel/air mixture, and so on.
- **Microprocessors** are usually classified according to their word length.
  - An 8-bit ***microcontroller*** is designed for low-cost applications and includes on-board memoryand I/O devices‘

- 16-bit microcontroller is often used for more sophisticated applications that may require either longer word lengths or off-chip I/O and memory;
- 32-bit ***RISC*** microprocessor offers very high performance for computation-intensive applications.

➢ **House Hold uses of microprocessor:**

- The typical **microwave oven** has at least one microprocessor to control oven operation.
- Many houses have **advanced thermostat systems**, which change the temperature level at various times during the day.

Characteristics of Embedded Computing Applications:

a. Complex Algorithms

b. User Interface

c. Real Time

d. Multirate

e.Manufacturing Cost

f. Power

Complex algorithms: The operations performed by the microprocessor may be very sophisticated. For example, the microprocessor that controls an automobile engine must perform complicated filtering functions to optimize the performance of the car while minimizing pollution and fuel utilization.

User interface: Microprocessors are frequently used to control complex user interfaces that may include multiple menus and many options. The moving maps in Global Positioning System (GPS) navigation are good examples of sophisticated user interfaces.

To make things more difficult, embedded computing operations must often be performed to meet deadlines:

Real time: Many embedded computing systems have to perform in real time— if the data is not ready by a certain deadline, the system breaks. In some cases, failure to meet a deadline is unsafe and can even endanger lives. In other cases, missing a deadline does not create safety problems but does create unhappy customers—missed deadlines in printers, for example, can result in scrambled pages.

➢ ***Multirate:*** Not only must operations be completed by deadlines, but many embedded computing systems have several real-time activities going on at the same time. They may simultaneously control some operations that run at slow rates and others that run at high rates. Multimedia applications are prime examples of ***multirate*** behaviour. The audio and video portions of a multimedia stream run at very different rates, but they must remain closely synchronized. Failure to meet a deadline on either the audio or video portions spoils the perception of the entire presentation.

Costs of various sorts are also very important:

➢ ***Manufacturing cost:*** The total cost of building the system is very important in many cases. Manufacturing cost is determined by many factors, including the type of microprocessor used, the amount of memory required, and the types of I/O devices.

➢ ***Power and energy:*** Power consumption directly affects the cost of the hardware, since a larger power supply may be necessary. Energy consumption affects battery life, which is important in many applications, as well as heat consumption, which can be important even in desktop applications.

Challenges in Embedded Computing System Design:

i. How much hardware do we need?
ii. How do we meet deadlines?
iii. How do we minimize power consumption?
iv. How do we design for upgradability?
v. Does it really work?
vi. Complex testing
vii. Limited observability and controllability
viii. Restricted development environments

External constraints are one important source of difficulty in embedded system design. Let's consider some important problems that must be taken into account in embedded system design.

How much hardware do we need?

We have a great deal of control over the amount of computing power we apply to our problem. We cannot only select the type of microprocessor used, but also select the amount of memory, the peripheral devices, and more. Since we often must meet both performance deadlines and manufacturing cost constraints, the choice of hardware is important—too little hardware and the system fails to meet its deadlines, too much hardware and it becomes too expensive.

How do we meet deadlines?

The brute force way of meeting a deadline is to speed up the hardware so that the program runs faster. Of course, that makes the system more expensive. It is also entirely possible that increasing the CPU clock rate may not make enough difference to execution time, since the program's speed may be limited by the memory system.

How do we minimize power consumption?

In battery-powered applications, power consumption is extremely important. Even in non battery applications, excessive power consumption can increase heat dissipation. One way to make a digital system consume less power is to make it run more slowly, slowing down the system can obviously lead to missed deadlines. Careful design is required to slow down the noncritical parts of the machine for power consumption while still meeting necessary performance goals.

How do we design for upgradability?

The hardware platform may be used over several product generations or for several different versions of a product in the same generation, with few or no changes. However, we want to be able to add features by changing software.

Does it really work?

Reliability is always important when selling products—customers rightly expect that products they buy will work. Reliability is especially important in some applications. If we wait until we have a running system and try to eliminate the bugs, we will be too late—we won't find enough bugs, it will be too expensive to fix them, and it will take more time.

Let's consider some ways in which the nature of embedded computing machines makes their design more difficult.

Complex testing: Exercising an embedded system is generally more difficult than typing in some data. We may have to run a real machine in order to generate the proper data. The timing of data is often important, meaning that we cannot separate the testing of an embedded computer from the machine in which it is embedded.

Limited observability and controllability: Embedded computing systems usually do not come with keyboards and screens. This makes it more difficult to see what is going on and to affect the system's operation. We may be forced to watch the values of electrical signals on the microprocessor bus, for example, to know what is going on inside the system. Moreover, in real-time applications we may not be able to easily stop the system to see what is going on inside.

Restricted development environments: The development environments for embedded systems (the tools used to develop software and hardware) are often much more limited than those available for PCs and workstations.

Embedded system design process:

This section provides an overview of the embedded system design process aimed at two objectives. First, it will give us an introduction to the various steps in embedded system design before we delve into them in more detail. Second, it will allow us to consider the design methodology itself. A design methodology is important for three reasons. First, it allows us to keep a scorecard on a design to ensure that we have done everything we need to do, such as optimizing performance or performing functional tests. Second, it allows us to develop computer-aided design tools. Developing a single program that takes in a concept for an embedded system and emits a completed design would be a daunting task, but by first breaking the process into manageable steps, we can work on automating (or at least semi automating) the steps one at a time. Third, a design methodology makes it much easier for members of a design team to communicate.

The below Figure summarizes the major steps in the embedded system design process. In this top–downview, we start with the system requirements.

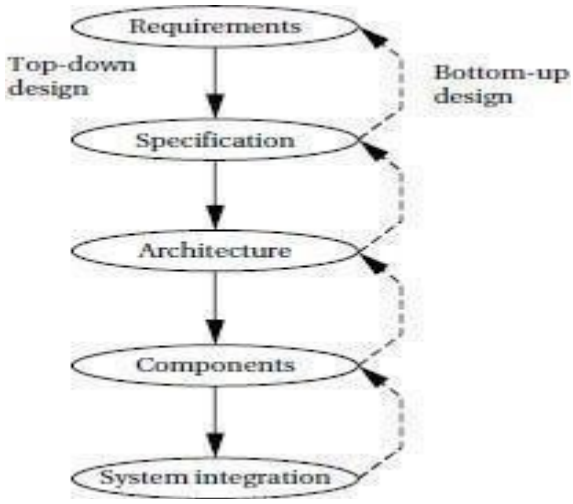


Fig: Major levels of abstraction in the design process

Requirements:

Clearly, before we design a system, we must know what we are designing. The initial stages of the design process capture this information for use in creating the architecture and components. We generally proceed in two phases: First, we gather an informal description from the customers known as requirements, and we refine the requirements into a specification that contains enough information to begin designing the system architecture.

Requirements may be ***functional*** or ***nonfunctional***. We must of course capture the basic functions of the embedded system, but functional description is often not sufficient. Typical nonfunctional requirements include:

■ *Performance:* The speed of the system is often a major consideration both for the usability of the system and for its ultimate cost. As we have noted, performance may be a combination of soft performance metrics such as approximate time to perform a user-level function and hard deadlines by which a particular operation must be completed.

■ *Cost:* The target cost or purchase price for the system is almost always a consideration. Cost typically has two major components: ***manufacturing cost*** includes the cost of components and assembly; ***nonrecurring engineering*** **(NRE)** costs include the personnel and other costs of designing the system.
■ *Physical size and weight:* The physical aspects of the final system can vary greatly depending upon the application. An industrial control system for an assembly line may be designed to fit into a standard-size rack with no strict limitations on weight. A handheld device typically has tight requirements on both size and weight that can ripple through the entire system design.
■ *Power consumption:* Power, of course, is important in battery-powered systems and is often important in other applications as well. Power can be specified in the requirements stage in terms of battery life—the customer is unlikely to be able to describe the allowable wattage.
A sample ***requirements form*** that can be filled out at the start of the project. We can use the form as a checklist in considering the basic characteristics of the system. Let‘s consider the entries in the form:
■ *Name:* This is simple but helpful. Giving a name to the project not only simplifies talking about it to other people but can also crystallize the purpose of the machine.
■ *Purpose:* This should be a brief one- or two-line description of what the system is supposed to do. If you can‘t describe the essence of your system in one or two lines, chances are that you don‘t understand it well enough.
■ *Inputs and outputs:* These two entries are more complex than they seem. The inputs and outputs to the system encompass a wealth of detail:
— *Types of data:* Analog electronic signals? Digital data? Mechanical inputs?
— *Data characteristics:* Periodically arriving data, such as digital audio
samples? Occasional user inputs? How many bits per data element?
— *Types of I/O devices:* Buttons? Analog/digital converters? Video displays?
■ *Functions:* This is a more detailed description of what the system does. A good way to approach this is to work from the inputs to the outputs: When the system receives an input, what does it do? How do user interface inputs affect these functions? How do different functions interact?
Performance: Many embedded computing systems spend at least some time controlling physical devices or processing data coming from the physical world. In most of these cases, the computations must be performed within a certain time frame. It is essential that the performance requirements be identified early
since they must be carefully measured during implementation to ensure that the system works properly.
■ *Manufacturing cost:* This includes primarily the cost of the hardware components. Even if you don‘t know exactly how much you can afford to spend on system components, you should have some idea of the eventual cost range. Cost has a substantial influence on architecture: A machine that is meant to
sell at $10 most likely has a very different internal structure than a $100 system.
■ *Power:* Similarly, you may have only a rough idea of how much power the system can consume, but a little information can go a long way. Typically, the most important decision is

---

whether the machine will be battery powered or plugged into the wall. Battery-powered machines must be much more careful about how they spend energy.

■ *Physical size and weight:* You should give some indication of the physical size of the system to help guide certain architectural decisions. A desktop machine has much more flexibility in the components used than, for example, a lapel mounted voice recorder.

GPS MODULE:



REQUIREMENTS FORM OF GPS MOVING MAP MODULE:

Name : GPS moving map
Purpose: Consumer-grade moving map for driving use
Inputs : Power button, two control buttons
Outputs : Back-lit LCD display 400 _ 600
Functions : Uses 5-receiver GPS system; three user-selectable resolutions; always displays current latitude and longitude
Performance: Updates screen within 0.25 seconds upon movement
Manufacturing cost:$30
Power: 100mW
Physical size and weight: No more than 2" _ 6, " 12 ounces

Specification

The specification is more precise—it serves as the contract between the customer and the architects. As such, the specification must be carefully written so that it accurately reflects the customer's requirements and does so in a way that can be clearly followed during design.

The specification should be understandable enough so that someone can verify that it meets system requirements and overall expectations of the customer.

A specification of the GPS system would include several components:

- Data received from the GPS satellite constellation.
- Map data.
- User interface.
- Operations that must be performed to satisfy customer requests.
- Background actions required to keep the system running, such as operating the GPS receiver.

Architecture Design

The specification does not say how the system does things, only what the system does. Describing how the system implements those functions is the purpose of the architecture. The architecture is a plan for the overall structure of the system that will be used later to design the components that make up the architecture. The creation of the architecture is the first phase of what many designers think of as design.

This block diagram is still quite abstract—we have not yet specified which operations will be performed by software running on a CPU, what will be done by special-purpose hardware, and so on. The diagram does, however, go a long way toward describing how to implement the functions described in the specification. We clearly see, for example, that we need to search the topographic database and to render (i.e., draw) the results for the display. We have chosen to separate those functions so that we can potentially do them in parallel—performing rendering separately from searching the database may help us update the screen more fluidly.

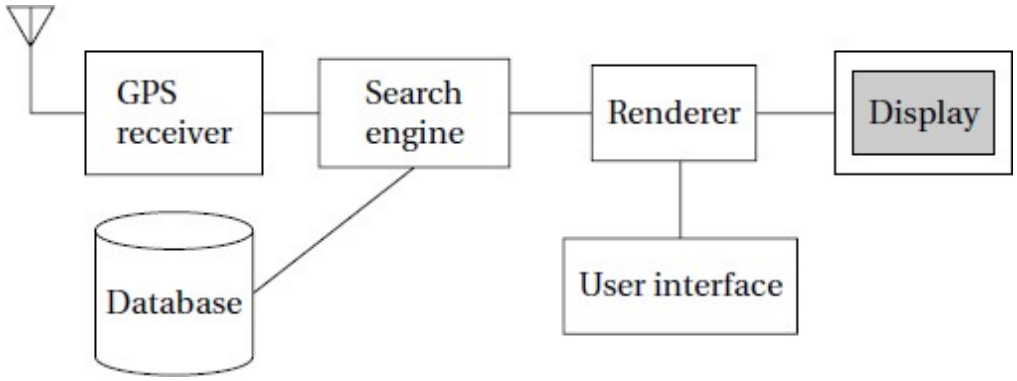


FIG: BLOCK DIAGRAM FOR THE MOVING MAP

The hardware block diagram clearly shows that we have one central CPU surrounded by memory and I/O devices. In particular, we have chosen to use two memories: a frame buffer for the pixels to be displayed and a separate program/data memory for general use by the CPU. The software block diagram fairly closely follows the system block diagram, but we have added a timer to control when we read the buttons on the user interface and render data onto the screen. To have a truly complete architectural description, we require more detail, such as where units in the software block diagram will be executed in the hardware block diagram and when operations will be performed in time.

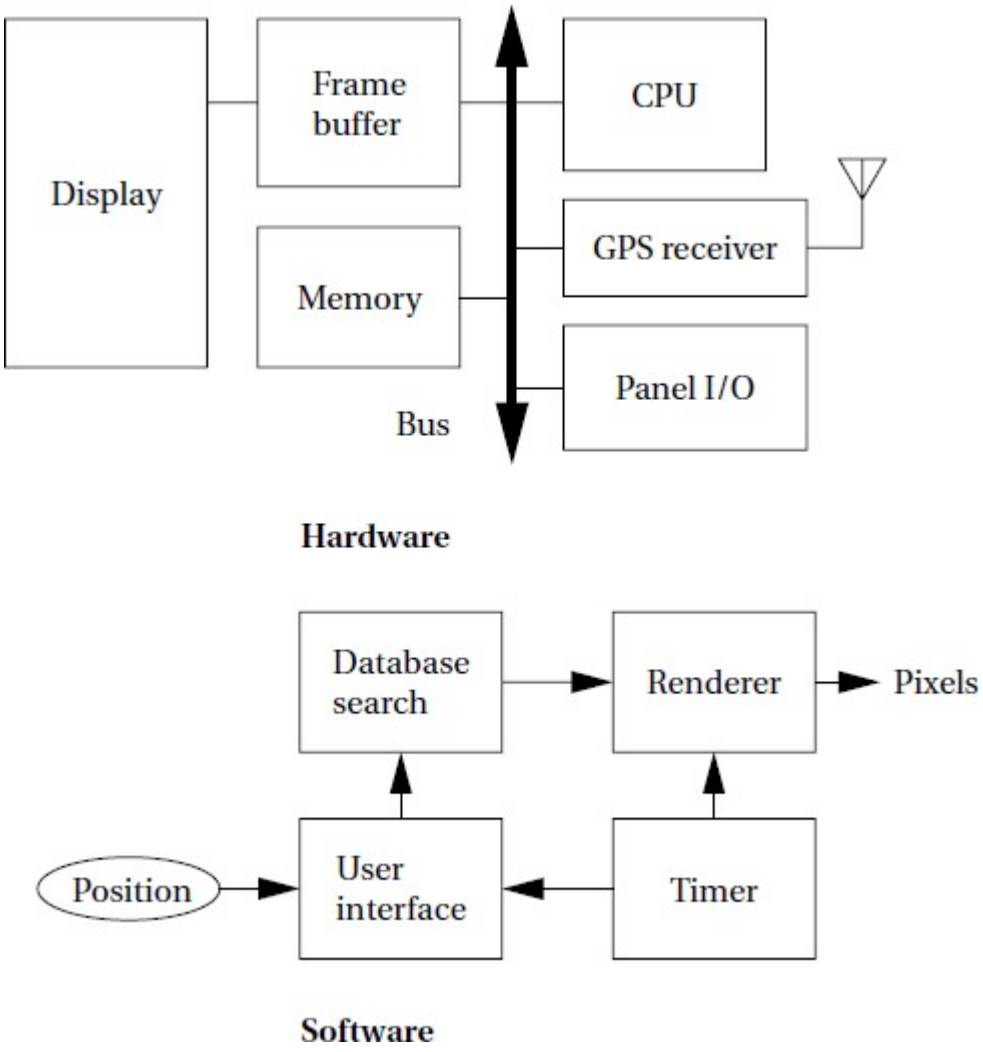


Fig : Hardware and software architectures for the moving map.

The architectural description tells us what components we need. The component design effort builds those components in conformance to the architecture and specification. The components will in general include both hardware—FPGAs, boards, and so on—and software modules. Some of the components will be ready-made. The CPU, for example, will be a standard component in almost all cases, as will memory chips and many other components .In the moving map, the GPS receiver is a good example of a specialized component that will nonetheless be a predesigned, standard component. We can also make use of standard software modules.

System Integration:

Only after the components are built do we have the satisfaction of putting them together and seeing a working system. Of course, this phase usually consists of a lot more than just plugging everything together and standing back. Bugs are typically found during system integration, and good planning can help us find the bugs quickly. By building up the system in phases and running properly chosen tests, we can often find bugs more easily. If we debug only a few modules at a time, we are more likely to uncover the simple bugs and able to easily recognize them. Only by fixing the simple bugs early will we be able to uncover the more complex or obscure bugs that can be identified only by giving the system a hard workout

- The **modern camera** is a prime example of the powerful features that can be added under microprocessor control.
- **Digital Television** uses embedded processors

DESIGN EXAMPLE:

BMW 850i brake and stability control system

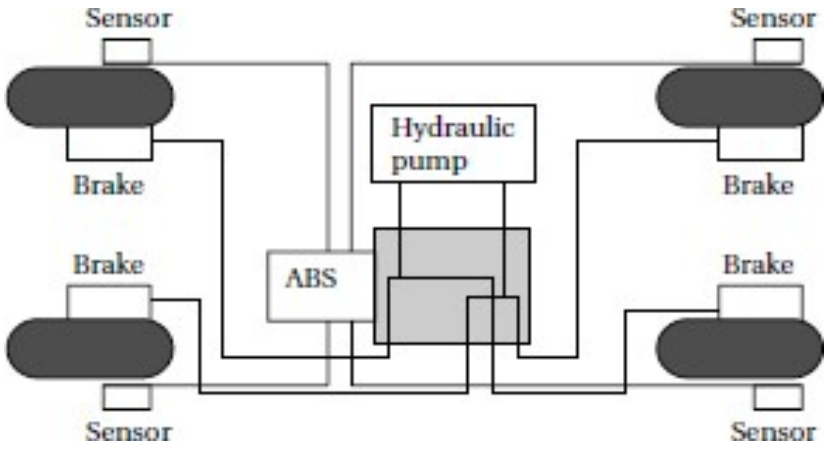
The BMW 850i was introduced with a sophisticated system for controlling the wheels of the car.

An antilock brake system (ABS) reduces skidding by pumping the brakes. An automatic stability control (ASC _ T) system intervenes with the engine during maneuvering to improve the car‘s stability.

These systems actively control critical systems of the car; as control systems, they require inputs from and output to the automobile.

Let‘s first look at the ABS. The purpose of an ABS is to temporarily release the brake on a wheel when it rotates too slowly—when a wheel stops turning, the car starts skidding and becomes hard to control.

It sits between the hydraulic pump, which provides power to the brakes, and the brakesthemselves as seen in the below diagram. The ABS system uses sensors on each wheel to measure the speed of the wheel. The wheel speeds are used by the ABS system to determine how to vary the hydraulic fluid pressure to prevent the wheels from skidding.



➢ The ASC _ T system‘s job is to control the engine power and the brake to improve the car‘s stability. The ASC _ T controls four different systems: throttle, ignition timing, differential brake, and (on automatic transmission cars) gear shifting.

SUMMARY

1. An embedded system is an electronic/electromechanical system designed to perform a specific function and is a combination of both hardware and firmware (software).
2. A general purpose computing system is a combination of generic hardware and general purpose operating system for executing a variety of applications, whereas an embedded
3. System is a combination of special purpose hardware and embedded OS/firmware for executing a specific set of applications.
4. Apollo Guidance Computer (AGC) is the first recognized modern embedded system and Autonetics D-17, the guidance computer for the Minuteman-I missile, was the first mass produced embedded system.
5. Based on the complexity and performance requirements, embedded systems are classified into small-scale, medium-scale and large-scale/complex.
6. The presences of embedded system vary from simple electronic system toys to complex flight and missile control systems.
7. Embedded systems are designed to serve the purpose of any one or combination of data collection/storage/representation, data processing, monitoring, control or application specific user interface.
8. Wearable devices refer to embedded systems which are incorporated into accessories and apparels. It envisions the bonding of embedded technology in our day to day lives.

OBJECTIVE QUESTIONS

1. Embedded systems are

   (a) General Purpose (b) Special Purpose
2. Embedded system is

   (a) An electronic system (b) A pure mechanical system

   (c)An electro-mechanical system (d) (a) or (c)
3. Which of the following is not true about embedded systems?

   (a) Built around specialized hardware (b) Always contain an operating system

   (c)Execution behavior may be deterministic (d) All of these (e) none of these
4. Which of the following is not an example of small scale embedded system?

   (a) Electronic Barbie doll (b) Simple calculator

   (c) Cell Phone (d) Electronic toy car

5. The first recognized modern embedded system is
   (a) Apple computer (b) Apollo Guidance Computer
   (c) Calculator (d) Radio navigation system
6. The first mass produced embedded system is
   (a) Minuteman-I (b) Minuteman-II
   (c) Autonetics D17 (d) Apollo Guidance Computer
7. Which of the following is (are) an intended purpose of embedded systems?
   (a) Data collection (b) Data processing
   (c) Data communication (d) All of these (e) None of these
8. Which of the following is an example of an embedded system for data communication?
   (a) USB mass storage device (b) Network router (c) Digital camera
   (d)Music player (e) All of these (f) None of these
9. A digital multimeter is an example of embedded system for
   (a) Data communication (b) Monitoring (c) Control
   (d) All of these (e) None of these
10. Which of the following is an example of an embedded system for signal processing?
    (a) Apple iPOD (b) Sandisk USB mass storage device
    (c) both a and b (d) None of these

Reference Text Books:-

1. Introduction to Embedded Systems – Shibu K.V Mc Graw Hill

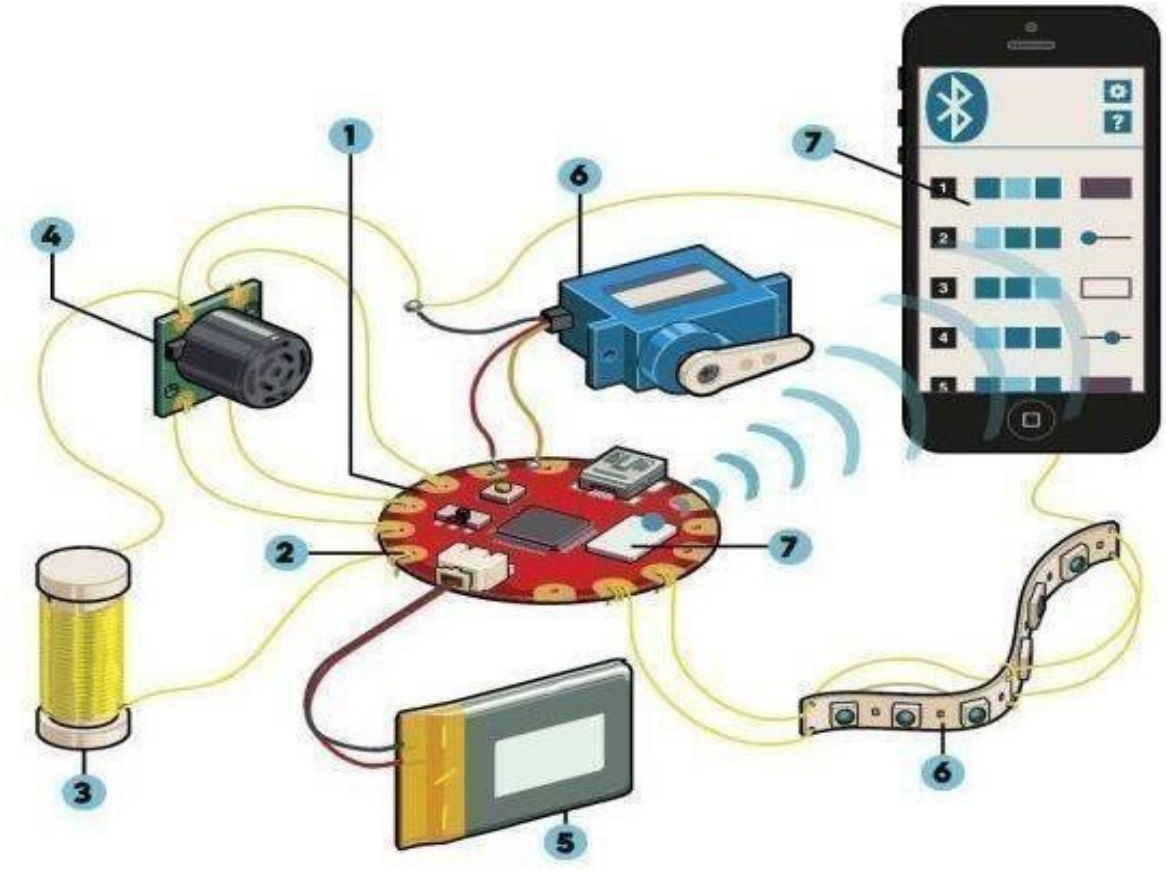
2. Computers as Components –Wayne Wolf-morgan Kaufmann publications

IV ECE

EMBEDDED SYSTEM DESIGN

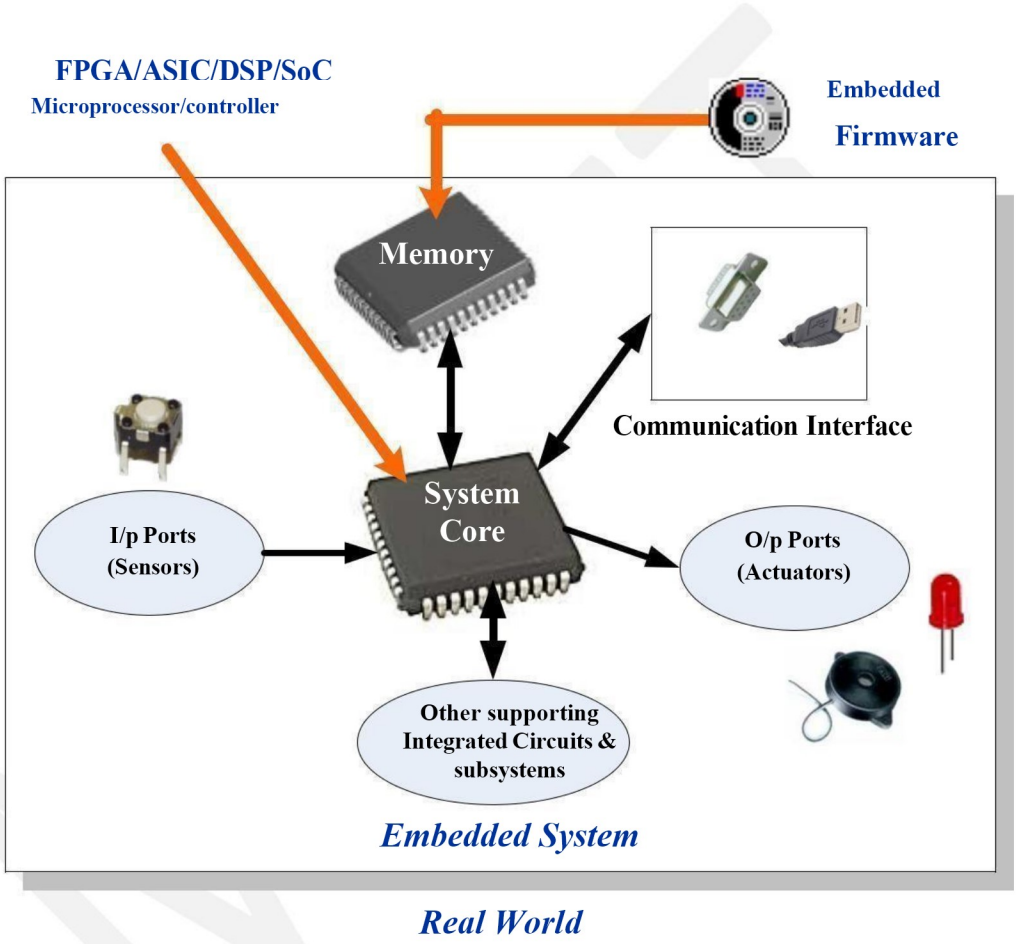
UNIT-II

TYPICAL EMBEDDED SYSTEM



ELEMENTS OF EMBEDDED SYSTEMS:

An embedded system is a combination of 3 things, Hardware Software Mechanical Components and it is supposed to do one specific task only. A typical embedded system contains a single chip controller which acts as the master brain of the system. Diagrammatically an embedded system can be represented as follows:



Embedded systems are basically designed to regulate a physical variable (such Microwave Oven) or to manipulate the state of some devices by sending some signals to the actuators or devices connected to the output port system (such as temperature in Air Conditioner), in response to the input signal provided by the end users or sensors which are connected to the input ports. Hence the embedded systems can be viewed as a reactive system.

The control is achieved by processing the information coming from the sensors and user interfaces and controlling some actuators that regulate the physical variable.

Keyboards, push button, switches, etc. are Examples of common user interface input devices and LEDs, LCDs, Piezoelectric buzzers, etc examples for common user interface output devices for a typical embedded system.The requirement of type of user interface changes from application to application based on domain.

Some embedded systems do not require any manual intervention for their operation. They automatically sense the input parameters from real world through sensors which are connected at input port. The sensor information is passed to the processor after signal conditioning and digitization. The core of the system performs some predefined operations on input data with the help of embedded firmware in the system and sends some actuating signals to the actuator connect connected to the output port of the system.

The memory of the system is responsible for holding the code (control algorithm and other important configuration details). There are two types of memories are used in any embedded system. Fixed memory (ROM) is used for storing code or program. The user cannot change the firmware in this type of memory. The most common types of memories used in embedded systems for control algorithm storage are OTP,PROM,UVEPROM,EEPROM and FLASH

An embedded system without code (i.e. the control algorithm) implemented memory has all the peripherals but is not capable of making decisions depending on the situational as well as real world changes.

Memory for implementing the code may be present on the processor or may be implemented as a separate chip interfacing the processor

In a controller based embedded system, the controller may contain internal memory for storing code such controllers are called Micro-controllers with on-chip ROM, eg. Atmel AT89C51.

The Core of the Embedded Systems: The core of the embedded system falls into anyone of the following categories.

- **General Purpose and Domain Specific Processors**
  - Microprocessors
  - Microcontrollers
  - Digital Signal Processors
- **Programmable Logic Devices (PLDs)**
- **Application Specific Integrated Circuits (ASICs)**
- **Commercial off the shelf Components (COTS)**

GENERAL PURPOSE AND DOMAIN SPECIFIC PROCESSOR:

Almost 80% of the embedded systems are processor/ controller based.

- The processor may be microprocessor or a microcontroller or digital signal processor,
- depending on the domain and application.

Microprocessor:

- A silicon chip representing a Central Processing Unit (CPU), which is capable of performing arithmetic as well as logical operations according to a pre-defined set of Instructions, which is specific to the manufacturer
- In general the CPU contains the Arithmetic and Logic Unit (ALU), Control Unit and Working registers
- Microprocessor is a dependant unit and it requires the combination of other hardware like Memory, Timer Unit, and Interrupt Controller etc for proper functioning.
- Intel claims the credit for developing the first Microprocessor unit Intel 4004, a 4 bit processor which was released in Nov 1971

· Developers of microprocessors.

- Intel – Intel 4004 – November 1971(4-bit)
- Intel – Intel 4040.
- Intel – Intel 8008 – April 1972.
- Intel – Intel 8080 – April 1974(8-bit).
- Motorola – Motorola 6800.
- Intel – Intel 8085 – 1976.
- Zilog - Z80 – July 1976

Microcontroller:

- A highly integrated silicon chip containing a CPU, scratch pad RAM, Special and General purpose Register Arrays, On Chip ROM/FLASH memory for program storage, Timer and Interrupt control units and dedicated I/O ports
- Microcontrollers can be considered as a super set of Microprocessors
- Microcontroller can be general purpose (like Intel 8051, designed for generic applications and domains) or application specific (Like Automotive AVR from Atmel Corporation. Designed specifically for automotive applications)
- Since a microcontroller contains all the necessary functional blocks for independent working, they found greater place in the embedded domain in place of microprocessors
- Microcontrollers are cheap, cost effective and are readily available in the market
- Texas Instruments TMS 1000 is considered as the world‟s first microcontroller

Microprocessor Vs Microcontroller:

Microprocessor	Microcontroller
A silicon chip representing a Central Processing Unit (CPU), which is capable of performing arithmetic as well as logical operations according to a pre-defined set of Instructions	A microcontroller is a highly integrated chip that contains a CPU, scratch pad RAM, Special and General purpose Register Arrays, On Chip ROM/FLASH memory for program storage, Timer and Interrupt control units and dedicated I/O ports
It is a dependent unit. It requires the combination of other chips like Timers, Program and data memory chips, Interrupt controllers etc for functioning	It is a self contained unit and it doesn‟t require external Interrupt Controller, Timer, UART etc for its functioning
Most of the time general purpose in design and operation	Mostly application oriented or domain specific
Doesn‟t contain a built in I/O port. The I/O Port functionality needs to be implemented with the help of external Programmable Peripheral Interface Chips like 8255	Most of the processors contain multiple built-in I/O ports which can be operated as a single 8 or 16 or 32 bit Port or as individual port pins
Targeted for high end market where performance is important	Targeted for embedded market where performance is not so critical (At present this demarcation is invalid)
Limited power saving options compared to microcontrollers	Includes lot of power saving features

General Purpose Processor (GPP) Vs Application Specific Instruction Set Processor (ASIP)

- General Purpose Processor or GPP is a processor designed for general computational tasks
- GPPs are produced in large volumes and targeting the general market. Due to the high volume production, the per unit cost for a chip is low compared to ASIC or other specific ICs
- A typical general purpose processor contains an Arithmetic and Logic Unit (ALU) and Control Unit (CU)
- Application Specific Instruction Set processors (ASIPs) are processors with architecture and instruction set optimized to specific domain/application requirements like Network processing, Automotive, Telecom, media applications, digital signal processing, control applications etc.
- ASIPs fill the architectural spectrum between General Purpose Processors and Application Specific Integrated Circuits (ASICs)
- The need for an ASIP arises when the traditional general purpose processor are unable to meet the increasing application needs
- Some Microcontrollers (like Automotive AVR, USB AVR from Atmel), System on Chips, Digital Signal Processors etc are examples of Application Specific Instruction Set Processors (ASIPs)
- ASIPs incorporate a processor and on-chip peripherals, demanded by the application requirement, program and data memory

Digital Signal Processors (DSPs):

- Powerful special purpose 8/16/32 bit microprocessors designed specifically to meet the computational demands and power constraints of today's embedded audio, video, and communications applications
- Digital Signal Processors are 2 to 3 times faster than the general purpose microprocessors in signal processing applications
- DSPs implement algorithms in hardware which speeds up the execution whereas general purpose processors implement the algorithm in firmware and the speed of execution depends primarily on the clock for the processors
- DSP can be viewed as a microchip designed for performing high speed computational operations for „addition", „subtraction", „multiplication" and „division"

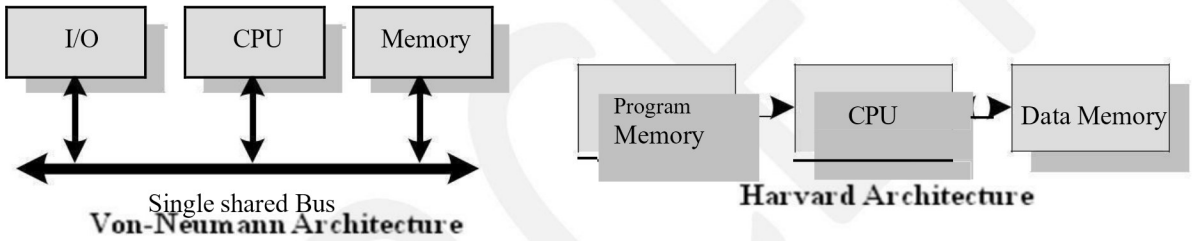
- A typical Digital Signal Processor incorporates the following key units
  - Program Memory
  - Data Memory
  - Computational Engine
  - I/O Unit
- Audio video signal processing, telecommunication and multimedia applications are typical examples where DSP is employed

RISC V/s CISC Processors/Controllers:

RISC	CISC
Lesser no. of instructions	Greater no. of Instructions
Instruction Pipelining and increased execution speed	Generally no instruction pipelining feature
Orthogonal Instruction Set (Allows each instruction to operate on any register and use any addressing mode)	Non Orthogonal Instruction Set (All instructions are not allowed to operate on anyregister and use any addressing mode. It is instruction specific)
Operations are performed on registers only, the only memory operations are load and store	Operations are performed on registers or memory depending on the instruction
Large number of registers are available	Limited no. of general purpose registers
Programmer needs to write more code to execute a task since the instructions are simpler ones	. A programmer can achieve the desired functionality with a single instruction which in turn provides the effect of using more simpler single instructions in RISC
Single, Fixed length Instructions	Variable length Instructions
Less Silicon usage and pin count	More silicon usage since more additional decoder logic is required to implement the complex instruction decoding.
With Harvard Architecture	Can be Harvard or Von-Neumann Architecture

Harvard V/s Von-Neumann Processor/Controller Architecture

- The terms Harvard and Von-Neumann refers to the processor architecture design.
- Microprocessors/controllers based on the **Von-Neumann** architecture shares a single common bus for fetching both instructions and data. Program instructions and data are stored in a common main memory
- Microprocessors/controllers based on the **Harvard** architecture will have separate data bus and instruction bus. This allows the data transfer and program fetching to occur simultaneously on both buses
- With Harvard architecture, the data memory can be read and written while the program memory is being accessed. These separated data memory and code memory buses allow one instruction to execute while the next instruction is fetched (—Pre-fetching‖)



Harvard V/s Von-Neumann Processor/Controller Architecture:

Harvard Architecture	Von-Neumann Architecture
Separate buses for Instruction and Data fetching	Single shared bus for Instruction and Data fetching
Easier to Pipeline, so high performance can be achieved	Low performance Compared to Harvard Architecture
Comparatively high cost	Cheaper
No memoryalignment problems	Allows self modifying codes†
Since data memory and program memory are stored physically in different locations, no chances for accidental corruption of program memory	Since data memory and program memory are stored physically in same chip, chances for accidental corruption of program memory

Big-endian V/s Little-endian processors:

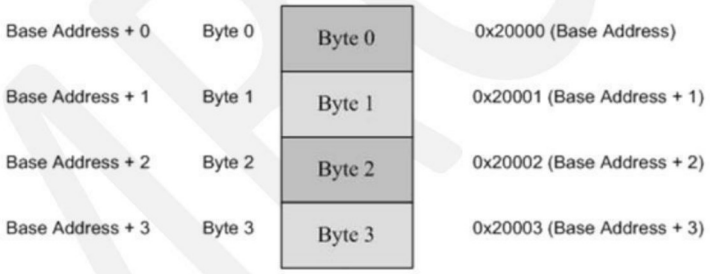
✓ Endianness specifies the order in which the data is stored in the memory by processor operations in a multi byte system (Processors whose word size is greater than one byte). Suppose the word length is two byte then data can be stored in memory in two different ways

- Higher order of data byte at the higher memory and lower order of data byte at location just below the higher memory
- Lower order of data byte at the higher memory and higher order of data byte at location just below the higher memory

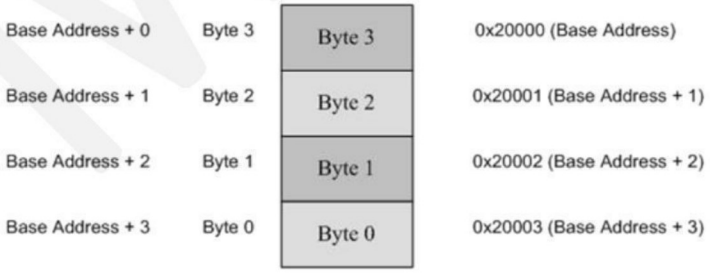
✓ *Little-endian* means the lower-order byte of the data is stored in memory at the lowest address, and the higher-order byte at the highest address. (The little end comes first)

✓ *Big-endian* means the higher-order byte of the data is stored in memory at the lowest address, and the lower-order byte at the highest address. (The big end comes first.)

Big-endian V/s Little-endian processors



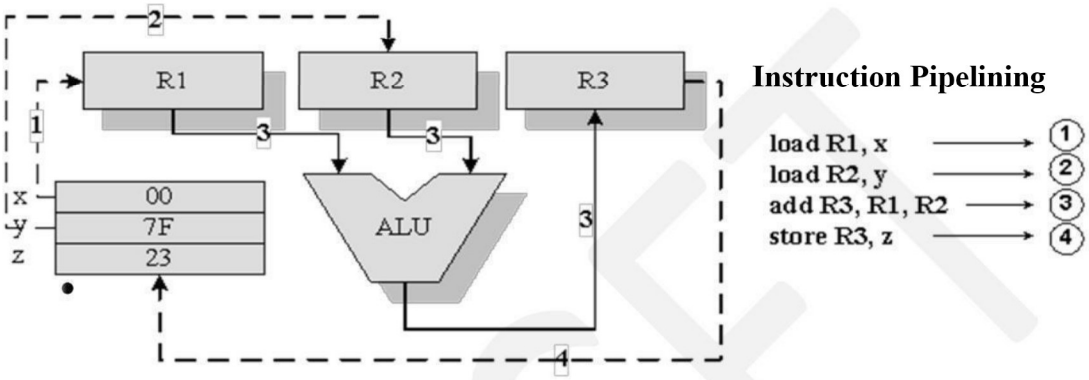
Little-endian Operation



Big-endian Operation

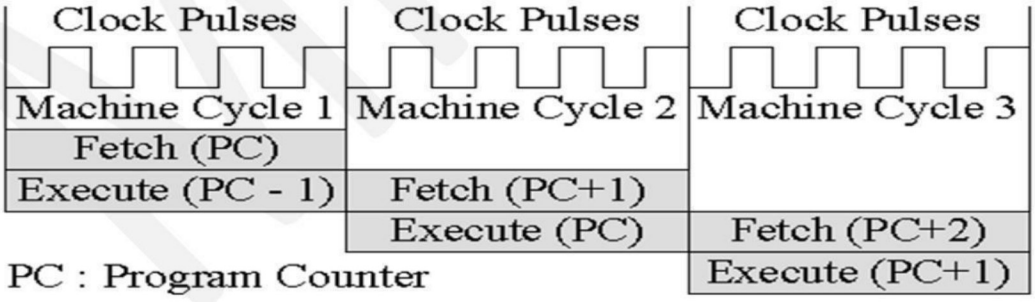
Load Store Operation & Instruction Pipelining:

The RISC processor instruction set is orthogonal and it operates on registers. The memory access related operations are performed by the special instructions *load* and *store*. If the operand is specified as memory location, the content of it is loaded to a register using the *load* instruction. The instruction *store* stores data from a specified register to a specified memory location



Load Store Operation

- The conventional instruction execution by the processor follows the fetch-decode-execute sequence
- The „fetch" part fetches the instruction from program memory or code memory and the decode part decodes the instruction to generate the necessary control signals



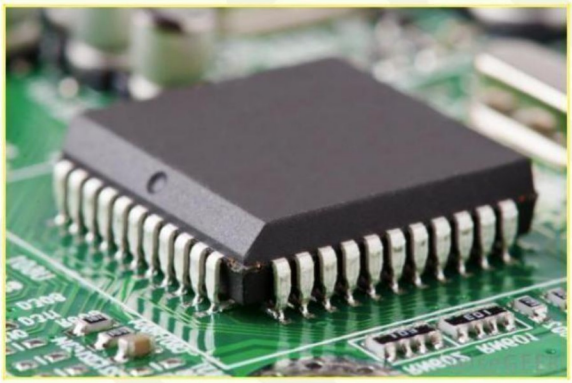
The Single stage pipelining concept

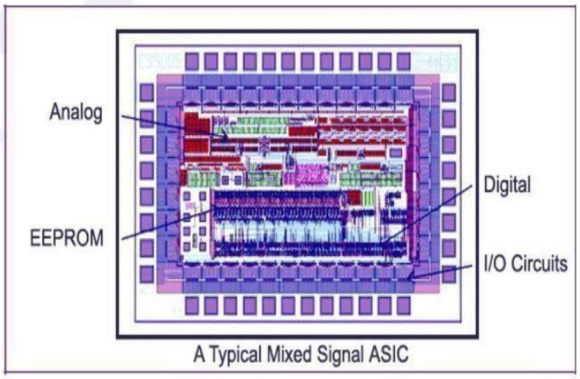
- The execute stage reads the operands, perform ALU operations and stores the result. In conventional program execution, the fetch and decode operations are performed in sequence

- During the decode operation the memory address bus is available and if it possible to effectively utilize it for an instruction fetch, the processing speed can be increased
- In its simplest form instruction pipelining refers to the overlapped execution of instructions

Application Specific Integrated Circuit (ASIC):

- A microchip designed to perform a specific or unique application. It is used as replacement to conventional general purpose logic chips.
- ASIC integrates several functions into a single chip and thereby reduces the system development cost
- Most of the ASICs are proprietary products. As a single chip, ASIC consumes very small area in the total system and thereby helps in the design of smaller systems with high capabilities/functionalities.
- ASICs can be pre-fabricated for a special application or it can be custom fabricated by using the components from a re-usable „*building block*" library of components for a particular customer application





A Typical Mixed Signal ASIC

- Fabrication of ASICs requires a non-refundable initial investment (Non Recurring Engineering (NRE) charges) for the process technology and configuration expenses
- If the Non-Recurring Engineering Charges (NRE) is born by a third party and the Application Specific Integrated Circuit (ASIC) is made openly available in the market, the ASIC is referred as Application Specific Standard Product (ASSP)
- The ASSP is marketed to multiple customers just as a general-purpose product , but to a smaller number of customers since it is for a specific application.

- Some ASICs are proprietary products , the developers are not interested in revealing the internal details.

Programmable Logic Devices (PLDs):

- Logic devices provide specific functions, including device-to-device interfacing, data communication, signal processing, data display, timing and control operations, and almost every other function a system must perform.
- Logic devices can be classified into two broad categories - Fixed and Programmable. The circuits in a fixed logic device are permanent, they perform one function or set of functions - once manufactured, they cannot be changed
- Programmable logic devices (PLDs) offer customers a wide range of logic capacity, features, speed, and voltage characteristics - and these devices can be re-configured to perform any number of functions at any time
- Designers can use inexpensive software tools to quickly develop, simulate, and test their logic designs in PLD based design. The design can be quickly programmed into a device, and immediately tested in a live circuit
- PLDs are based on re-writable memory technology and the device is reprogrammed to change the design

Programmable Logic Devices (PLDs) – CPLDs and FPGA

- Field Programmable Gate Arrays (FPGAs) and Complex Programmable Logic Devices (CPLDs) are the two major types of programmable logic devices

FPGA:

- FPGA is an IC designed to be configured bya designer after manufacturing.
- FPGAs offer the highest amount of logic density, the most features, and the highest performance.
- Logic gate is Medium to high density ranging from **1K to 500K** system gates

- These advanced FPGA devices also offer features such as built-in hardwired processors (such as the IBM Power PC), substantial amounts of memory, clock management systems, and support for many of the latest, very fast device-to-device signaling technologies

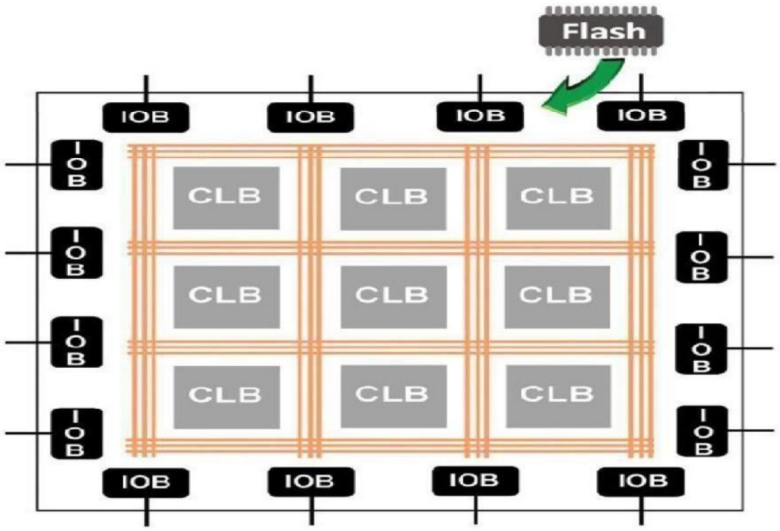


Figure: FPGA Architecture

- These advanced FPGA devices also offer features such as built-in hardwired processors, substantial amounts of memory, clock management systems, and support for many of the latest, very fast device-to-device signaling technologies.

- FPGAs are used in a wide variety of applications ranging from data processing and storage, to instrumentation, telecommunications, and digital signal processing

CPLD:

- A **complex programmable logic device** (**CPLD**) is a programmable logic device with complexity between that of PALs and FPGAs, and architectural features of both.

- CPLDs, by contrast, offer much smaller amounts of logic - up to about 10,000 gates.

- CPLDs offer very predictable timing characteristics and are therefore ideal for critical control applications.